

Our training environment



SSH terminal

Our training environment

- If you are attending a tutorial or workshop:
 - a VM has been provisioned for each student
- If you are doing or re-doing this course on your own, you can:
 - install Docker locally (as explained in the chapter “Installing Docker”)
 - install Docker on e.g. a cloud VM
 - use <https://www.play-with-docker.com/> to instantly get a training environment

This section assumes that you are following this course as part of a tutorial, training or workshop, where each student is given an individual Docker VM.

- The VM is created just before the training.
- It will stay up during the whole training.
- It will be destroyed shortly after the training.
- It comes pre-loaded with Docker and some other useful tools.

What is Docker?

- “Installing Docker” really means “Installing the Docker Engine and CLI”.
- The Docker Engine is a daemon (a service running in the background).
- This daemon manages containers, the same way that a hypervisor manages VMs.
- We interact with the Docker Engine by using the Docker CLI.
- The Docker CLI and the Docker Engine communicate through an API.
- There are many other programs and client libraries which use that API.

Why don't we run Docker locally?

- We are going to download container images and distribution packages.
- This could put a bit of stress on the local WiFi and slow us down.
- Instead, we use a remote VM that has a good connectivity
- In some rare cases, installing Docker locally is challenging:
 - no administrator/root access (computer managed by strict corp IT)
 - 32-bit CPU or OS
 - old OS version (e.g. CentOS 6, OSX pre-Yosemite, Windows 7)
- It's better to spend time learning containers than fiddling with the installer!

Connecting to your Virtual Machine

You need an SSH client.

- On OS X, Linux, and other UNIX systems, just use `ssh`:

```
$ ssh <login>@<ip-address>
```

- On Windows, if you don't have an SSH client, you can download:
 - Putty (www.putty.org)
 - Git BASH (<https://git-for-windows.github.io/>)
 - MobaXterm (<https://mobaxterm.mobatek.net/>)

The shell history of the instructor is available online in real time.

Note the IP address of the instructor's virtual machine (A.B.C.D).

Open `http://A.B.C.D:1088` in your browser and you should see the history.

The history is updated in real time (using a WebSocket connection).

It should be green when the WebSocket is connected.

If it turns red, reloading the page should fix it.

Checking your Virtual Machine

Once logged in, make sure that you can run a basic Docker command:

```
$ docker version
```

```
Client:
```

```
Version:      18.03.0-ce
API version:   1.37
Go version:    go1.9.4
Git commit:    0520e24
Built:         Wed Mar 21 23:10:06 2018
OS/Arch:       linux/amd64
Experimental:  false
Orchestrator:  swarm
```

```
Server:
```

```
Engine:
```

```
Version:      18.03.0-ce
API version:   1.37 (minimum version 1.12)
Go version:    go1.9.4
Git commit:    0520e24
Built:         Wed Mar 21 23:08:35 2018
OS/Arch:       linux/amd64
Experimental:  false
```

Installing Docker



install

At the end of this lesson, you will know:

- How to install Docker.
- When to use `sudo` when running Docker commands.

Note: if you were provided with a training VM for a hands-on tutorial, you can skip this chapter, since that VM already has Docker installed, and Docker has already been setup to run without `sudo`.

There are many ways to install Docker.

We can arbitrarily distinguish:

- Installing Docker on an existing Linux machine (physical or VM)
- Installing Docker on macOS or Windows
- Installing Docker on a fleet of cloud VMs

Installing Docker on Linux

- The recommended method is to install the packages supplied by Docker Inc :
 - add Docker Inc.'s package repositories to your system configuration
 - install the Docker Engine
- Detailed installation instructions (distro by distro) are available on:
<https://docs.docker.com/engine/installation/>
- You can also install from binaries (if your distro is not supported):
<https://docs.docker.com/engine/installation/linux/docker-ce/binaries/>
- To quickly setup a dev environment, Docker provides a convenience install script:

```
curl -fsSL get.docker.com | sh
```



Docker Inc. packages vs distribution packages

- Docker Inc. releases new versions monthly (edge) and quarterly (stable)
- Releases are immediately available on Docker Inc.'s package repositories
- Linux distros don't always update to the latest Docker version
(Sometimes, updating would break their guidelines for major/minor upgrades)
- Sometimes, some distros have carried packages with custom patches
- Sometimes, these patches added critical security bugs 😞
- Installing through Docker Inc.'s repositories is a bit of extra work ...
... but it is generally worth it!

Installing Docker on macOS and Windows

- On macOS, the recommended method is to use Docker Desktop for Mac:

<https://docs.docker.com/docker-for-mac/install/>

- On Windows 10 Pro, Enterprise, and Education, you can use Docker Desktop for Windows:

<https://docs.docker.com/docker-for-windows/install/>

- On older versions of Windows, you can use the Docker Toolbox:

https://docs.docker.com/toolbox/toolbox_install_windows/

- On Windows Server 2016, you can also install the native engine:

<https://docs.docker.com/install/windows/docker-ee/>



- Special Docker edition available for Mac and Windows
- Integrates well with the host OS:
 - installed like normal user applications on the host
 - provides user-friendly GUI to edit Docker configuration and settings
- Only support running one Docker VM at a time ...
... but we can use `docker-machine`, the Docker Toolbox, VirtualBox, etc. to get a cluster.

- Leverages the host OS virtualization subsystem
(e.g. the [Hypervisor API](#) on macOS)
- Under the hood, runs a tiny VM
(transparent to our daily use)
- Accesses network resources like normal applications
(and therefore, plays better with enterprise VPNs and firewalls)
- Supports filesystem sharing through volumes
(we'll talk about this later)

When you execute `docker version` from the terminal:

- the CLI connects to the Docker Engine over a standard socket,
- the Docker Engine is, in fact, running in a VM,
- ... but the CLI doesn't know or care about that,
- the CLI sends a request using the REST API,
- the Docker Engine in the VM processes the request,
- the CLI gets the response and displays it to you.

All communication with the Docker Engine happens over the API.

This will also allow to use remote Engines exactly as if they were local.

Important PSA about security

- If you have access to the Docker control socket, you can take over the machine
(Because you can run containers that will access the machine's resources)
- Therefore, on Linux machines, the `docker` user is equivalent to `root`
- You should restrict access to it like you would protect `root`
- By default, the Docker control socket belongs to the `docker` group
- You can add trusted users to the `docker` group
- Otherwise, you will have to prefix every `docker` command with `sudo`, e.g.:
`sudo docker version`

Our first containers



Colorful plastic tubs

Objectives

At the end of this lesson, you will have:

- Seen Docker in action.
- Started your first containers.

In your Docker environment, just run the following command:

```
$ docker run busybox echo hello world  
hello world
```

(If your Docker install is brand new, you will also see a few extra lines, corresponding to the download of the `busybox` image.)

That was our first container!

- We used one of the smallest, simplest images available: `busybox`.
- `busybox` is typically used in embedded systems (phones, routers...)
- We ran a single process and echo'ed `hello world`.

Let's run a more exciting container:

```
$ docker run -it ubuntu  
root@04c0bb0a6c07: /#
```

- This is a brand new container.
- It runs a bare-bones, no-frills `ubuntu` system.
- `-it` is shorthand for `-i -t`.
 - `-i` tells Docker to connect us to the container's stdin.
 - `-t` tells Docker that we want a pseudo-terminal.

Do something in our container

Try to run `figlet` in our container.

```
root@04c0bb0a6c07:/# figlet hello  
bash: figlet: command not found
```

Alright, we need to install it.

We want `figlet`, so let's install it:

```
root@04c0bb0a6c07:/# apt-get update
...
Fetched 1514 kB in 14s (103 kB/s)
Reading package lists... Done
root@04c0bb0a6c07:/# apt-get install figlet
Reading package lists... Done
...
```

One minute later, `figlet` is installed!

Try to run our freshly installed program

The `figlet` program takes a message as parameter.

```
root@04c0bb0a6c07:/# figlet hello
```

Beautiful! 🤗

Counting packages in the container

Let's check how many packages are installed there.

```
root@04c0bb0a6c07:/# dpkg -l | wc -l  
97
```

- `dpkg -l` lists the packages installed in our container
- `wc -l` counts them

How many packages do we have on our host?

Exit the container by logging out of the shell, like you would usually do.
(E.g. with `^D` or `exit`)

```
root@04c0bb0a6c07:/# exit
```

Now, try to:

- run `dpkg -l | wc -l`. How many packages are installed?
- run `figlet`. Does that work?

Comparing the container and the host

Exit the container by logging out of the shell, with `^D` or `exit`.

Now try to run `figlet`. Does that work?

(It shouldn't; except if, by coincidence, you are running on a machine where `figlet` was installed before.)



Host and containers are independent things

- We ran an `ubuntu` container on an Linux/Windows/macOS host.
- They have different, independent packages.
- Installing something on the host doesn't expose it to the container.
- And vice-versa.
- Even if both the host and the container have the same Linux distro!
- We can run *any container* on *any host*.
(One exception: Windows containers can only run on Windows hosts; at least for now.)

Where's our container?

- Our container is now in a *stopped* state.
- It still exists on disk, but all compute resources have been freed up.
- We will see later how to get back to that container.

What if we start a new container, and try to run `figlet` again?

```
$ docker run -it ubuntu  
root@b13c164401fb:/# figlet  
bash: figlet: command not found
```

- We started a *brand new container*.
- The basic Ubuntu image was used, and `figlet` is not here.

Where's my container?

- Can we reuse that container that we took time to customize?

We can, but that's not the default workflow with Docker.

- What's the default workflow, then?

Always start with a fresh container. If we need something installed in our container, build a custom image.

- That seems complicated!

We'll see that it's actually pretty easy!

- And what's the point?

This puts a strong emphasis on automation and repeatability. Let's see why ...

- In the “pets vs. cattle” metaphor, there are two kinds of servers.
- Pets:
 - have distinctive names and unique configurations
 - when they have an outage, we do everything we can to fix them
- Cattle:
 - have generic names (e.g. with numbers) and generic configuration
 - configuration is enforced by configuration management, golden images ...
 - when they have an outage, we can replace them immediately with a new server
- What's the connection with Docker and containers?

- When we use local VMs (with e.g. VirtualBox or VMware), our workflow looks like this:
 - create VM from base template (Ubuntu, CentOS...)
 - install packages, set up environment
 - work on project
 - when done, shut down VM
 - next time we need to work on project, restart VM as we left it
 - if we need to tweak the environment, we do it live
- Over time, the VM configuration evolves, diverges.
- We don't have a clean, reliable, deterministic way to provision that environment.

- With Docker, the workflow looks like this:
 - create container image with our dev environment
 - run container with that image
 - work on project
 - when done, shut down container
 - next time we need to work on project, start a new container
 - if we need to tweak the environment, we create a new image
- We have a clear definition of our environment, and can share it reliably with others.
- Let's see in the next chapters how to bake a custom image with figlet!

???

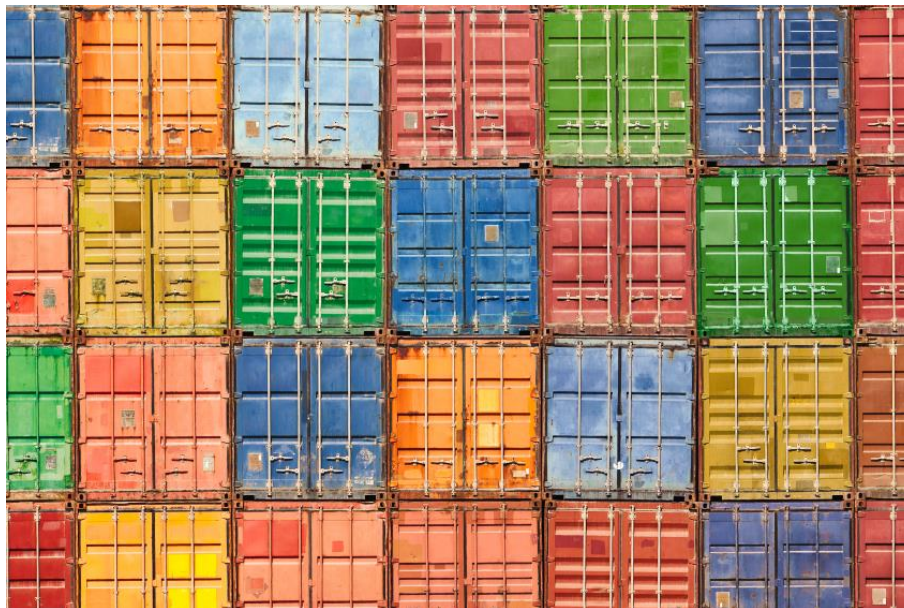


ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

:EN:- Running our first container :FR:- Lancer nos premiers conteneurs

Background containers



Background containers

Our first containers were *interactive*.

We will now see how to:

- Run a non-interactive container.
- Run a container in the background.
- List running containers.
- Check the logs of a container.
- Stop a container.
- List stopped containers.

A non-interactive container

We will run a small custom container.

This container just displays the time every second.

```
$ docker run jpetazzo/clock
Fri Feb 20 00:28:53 UTC 2015
Fri Feb 20 00:28:54 UTC 2015
Fri Feb 20 00:28:55 UTC 2015
...
```

- This container will run forever.
- To stop it, press `^C`.
- Docker has automatically downloaded the image `jpetazzo/clock`.
- This image is a user image, created by `jpetazzo`.



When ^C doesn't work...

Sometimes, ^C won't be enough.

Why? And how can we stop the container in that case?

What happens when we hit ^C

`SIGINT` gets sent to the container, which means:

- `SIGINT` gets sent to PID 1 (default case)
- `SIGINT` gets sent to *foreground processes* when running with `-ti`

But there is a special case for PID 1: it ignores all signals!

- except `SIGKILL` and `SIGSTOP`
- except signals handled explicitly

TL,DR: there are many circumstances when ^C won't stop the container.

Why is PID 1 special?

- PID 1 has some extra responsibilities:
 - it starts (directly or indirectly) every other process
 - when a process exits, its processes are “reparented” under PID 1
- When PID 1 exits, everything stops:
 - on a “regular” machine, it causes a kernel panic
 - in a container, it kills all the processes
- We don't want PID 1 to stop accidentally
- That's why it has these extra protections

How to stop these containers, then?

- Start another terminal and forget about them
(for now!)
- We'll shortly learn about `docker kill`

Run a container in the background

Containers can be started in the background, with the `-d` flag (daemon mode):

```
$ docker run -d jpetazzo/clock  
47d677dcfba4277c6cc68fcaa51f932b544cab1a187c853b7d0  
caf4e8debe5ad
```

- We don't see the output of the container.
- But don't worry: Docker collects that output and logs it!
- Docker gives us the ID of the container.

List running containers

How can we check that our container is still running?

With `docker ps`, just like the UNIX `ps` command, lists running processes.

```
$ docker ps
CONTAINER ID   IMAGE                                ...   CREATED           STATUS
...
47d677dcfba4   jpetazzo/clock                     ...   2 minutes ago     Up 2
minutes ...
```

Docker tells us:

- The (truncated) ID of our container.
- The image used to start the container.
- That our container has been running (Up) for a couple of minutes.
- Other information (COMMAND, PORTS, NAMES) that we will explain later.

Let's start two more containers.

```
$ docker run -d jpetazzo/clock  
57ad9bdfc06bb4407c47220cf59ce21585dce9a1298d7a67488  
359aeaea8ae2a
```

```
$ docker run -d jpetazzo/clock  
068cc994ffd0190bbe025ba74e4c0771a5d8f14734af772ddee  
8dc1aaf20567d
```

Check that `docker ps` correctly reports all 3 containers.

Viewing only the last container started

When many containers are already running, it can be useful to see only the last container that was started.

This can be achieved with the `-l` (“Last”) flag:

```
$ docker ps -l
```

CONTAINER ID	IMAGE	...	CREATED
STATUS	...		
068cc994ffd0	jpetazzo/clock	...	2 minutes ago
Up 2 minutes	...		

View only the IDs of the containers

Many Docker commands will work on container IDs: `docker stop`, `docker rm...`

If we want to list only the IDs of our containers (without the other columns or the header line), we can use the `-q` (“Quiet”, “Quick”) flag:

```
$ docker ps -q
068cc994ffd0
57ad9bdfc06b
47d677dcfba4
```

We can combine `-l` and `-q` to see only the ID of the last container started:

```
$ docker ps -lq  
068cc994ffd0
```

At a first glance, it looks like this would be particularly useful in scripts. However, if we want to start a container and get its ID in a reliable way, it is better to use `docker run -d`, which we will cover in a bit.

(Using `docker ps -lq` is prone to race conditions: what happens if someone else, or another program or script, starts another container just before we run `docker ps -lq`?)

We told you that Docker was logging the container output.
Let's see that now.

```
$ docker logs 068
Fri Feb 20 00:39:52 UTC 2015
Fri Feb 20 00:39:53 UTC 2015
...
```

- We specified a *prefix* of the full container ID.
- You can, of course, specify the full ID.
- The `logs` command will output the *entire* logs of the container.
(Sometimes, that will be too much. Let's see how to address that.)

To avoid being spammed with eleventy pages of output, we can use the `--tail` option:

```
$ docker logs --tail 3 068  
Fri Feb 20 00:55:35 UTC 2015  
Fri Feb 20 00:55:36 UTC 2015  
Fri Feb 20 00:55:37 UTC 2015
```

- The parameter is the number of lines that we want to see.

Follow the logs in real time

Just like with the standard UNIX command `tail -f`, we can follow the logs of our container:

```
$ docker logs --tail 1 --follow 068
Fri Feb 20 00:57:12 UTC 2015
Fri Feb 20 00:57:13 UTC 2015
^C
```

- This will display the last line in the log file.
- Then, it will continue to display the logs in real time.
- Use `^C` to exit.

Stop our container

There are two ways we can terminate our detached container.

- Killing it using the `docker kill` command.
- Stopping it using the `docker stop` command.

The first one stops the container immediately, by using the `KILL` signal.

The second one is more graceful. It sends a `TERM` signal, and after 10 seconds, if the container has not stopped, it sends `KILL`.

Reminder: the `KILL` signal cannot be intercepted, and will forcibly terminate the container.

Let's stop one of those containers:

```
$ docker stop 47d6  
47d6
```

This will take 10 seconds:

- Docker sends the TERM signal;
- the container doesn't react to this signal (it's a simple Shell script with no special signal handling);
- 10 seconds later, since the container is still running, Docker sends the KILL signal;
- this terminates the container.

Killing the remaining containers

Let's be less patient with the two other containers:

```
$ docker kill 068 57ad  
068  
57ad
```

The `stop` and `kill` commands can take multiple container IDs.

Those containers will be terminated immediately (without the 10-second delay).

Let's check that our containers don't show up anymore:

```
$ docker ps
```

List stopped containers

We can also see stopped containers, with the `-a` (`--all`) option.

```
$ docker ps -a
```

CONTAINER ID	IMAGE	...	CREATED	STATUS
068cc994ffd0	jpetazzo/clock	...	21 min. ago	Exited (137)
3 min. ago				
57ad9bdfc06b	jpetazzo/clock	...	21 min. ago	Exited (137)
3 min. ago				
47d677dcfba4	jpetazzo/clock	...	23 min. ago	Exited (137)
3 min. ago				
5c1dfd4d81f1	jpetazzo/clock	...	40 min. ago	Exited (0)
40 min. ago				
b13c164401fb	ubuntu	...	55 min. ago	Exited (130)
53 min. ago				

Naming and inspecting containers



Markings on container door

In this lesson, we will learn about an important Docker concept: container *naming*.

Naming allows us to:

- Reference easily a container.
- Ensure unicity of a specific container.

We will also see the `inspect` command, which gives a lot of details about a container.

So far, we have referenced containers with their ID.
We have copy-pasted the ID, or used a shortened prefix.
But each container can also be referenced by its name.
If a container is named `thumbnail-worker`, I can do:

```
$ docker logs thumbnail-worker  
$ docker stop thumbnail-worker  
etc.
```

Default names

When we create a container, if we don't give a specific name, Docker will pick one for us.

It will be the concatenation of:

- A mood (furious, goofy, suspicious, boring...)
- The name of a famous inventor (tesla, darwin, wozniak...)

Examples: `happy_curie`, `clever_hopper`, `jovial_lovelace` ...

Specifying a name

You can set the name of the container when you create it.

```
$ docker run --name ticktock jpetazzo/clock
```

If you specify a name that already exists, Docker will refuse to create the container.

This lets us enforce unicity of a given resource.

Renaming containers

- You can rename containers with `docker rename`.
- This allows you to “free up” a name without destroying the associated container.

The `docker inspect` command will output a very detailed JSON map.

```
$ docker inspect <containerID>
[ {
...
(many pages of JSON here)
...
}
```

There are multiple ways to consume that information.

- You *could* grep and cut or awk the output of `docker inspect`.
- Please, don't.
- It's painful.
- If you really must parse JSON from the Shell, use JQ! (It's great.)

```
$ docker inspect <containerID> | jq .
```
- We will see a better solution which doesn't require extra tools.

You can specify a format string, which will be parsed by Go's text/template package.

```
$ docker inspect --format '{{ json .Created }}'
<containerID>
"2015-02-24T07:21:11.712240394Z"
```

- The generic syntax is to wrap the expression with double curly braces.
- The expression starts with a dot representing the JSON object.
- Then each field or member can be accessed in dotted notation syntax.
- The optional `json` keyword asks for valid JSON output. (e.g. here it adds the surrounding double-quotes.)

Getting inside a container



Person standing inside a container

On a traditional server or VM, we sometimes need to:

- log into the machine (with SSH or on the console),
- analyze the disks (by removing them or rebooting with a rescue system).

In this chapter, we will see how to do that with containers.

Every once in a while, we want to log into a machine.

In an perfect world, this shouldn't be necessary.

- You need to install or update packages (and their configuration)?
Use configuration management. (e.g. Ansible, Chef, Puppet, Salt...)
- You need to view logs and metrics?
Collect and access them through a centralized platform.

In the real world, though ... we often need shell access!

Not getting a shell

Even without a perfect deployment system, we can do many operations without getting a shell.

- Installing packages can (and should) be done in the container image.
- Configuration can be done at the image level, or when the container starts.
- Dynamic configuration can be stored in a volume (shared with another container).
- Logs written to stdout are automatically collected by the Docker Engine.
- Other logs can be written to a shared volume.
- Process information and metrics are visible from the host.

Let's save logging, volumes ... for later, but let's have a look at process information!

Viewing container processes from the host

If you run Docker on Linux, container processes are visible on the host.

```
$ ps faux | less
```

- Scroll around the output of this command.
- You should see the `jpetazzo/clock` container.
- A containerized process is just like any other process on the host.
- We can use tools like `lsof`, `strace`, `gdb` ... To analyze them.

What's the difference between a container process and a host process?

- Each process (containerized or not) belongs to *namespaces* and *cgroups*.
- The namespaces and cgroups determine what a process can “see” and “do”.
- Analogy: each process (containerized or not) runs with a specific UID (user ID).
- UID=0 is root, and has elevated privileges. Other UIDs are normal users.

We will give more details about namespaces and cgroups later.

Getting a shell in a running container

- Sometimes, we need to get a shell anyway.
- We *could* run some SSH server in the container ...
- But it is easier to use `docker exec`.
\$ `docker exec -ti ticktock sh`
- This creates a new process (running `sh`) *inside* the container.
- This can also be done “manually” with the tool `nsenter`.

- The tool that you want to run needs to exist in the container.
- Some tools (like `ip netns exec`) let you attach to *one* namespace at a time.
(This lets you e.g. setup network interfaces, even if you don't have `ifconfig` or `ip` in the container.)
- Most importantly: the container needs to be running.
- What if the container is stopped or crashed?

Getting a shell in a stopped container

- A stopped container is only *storage* (like a disk drive).
- We cannot SSH into a disk drive or USB stick!
- We need to connect the disk to a running machine.
- How does that translate into the container world?

Analyzing a stopped container

As an exercise, we are going to try to find out what's wrong with `jpetazzo/crashtest`.

```
docker run jpetazzo/crashtest
```

The container starts, but then stops immediately, without any output.

What would MacGyver™ do?

First, let's check the status of that container.

```
docker ps -l
```

- We can use `docker diff` to see files that were added / changed / removed.

```
docker diff <container_id>
```

- The container ID was shown by `docker ps -l`.
- We can also see it with `docker ps -lq`.
- The output of `docker diff` shows some interesting log files!

- We can extract files with `docker cp`.

```
docker cp <container_id>:/var/log/nginx/error.log .
```

- Then we can look at that log file.

```
cat error.log
```

(The directory `/run/nginx` doesn't exist.)

Exploring a crashed container

- We can restart a container with `docker start ...`
- ... But it will probably crash again immediately!
- We cannot specify a different program to run with `docker start`
- But we can create a new image from the crashed container

```
docker commit <container_id> debugimage
```

- Then we can run a new container from that image, with a custom entrypoint

```
docker run -ti --entrypoint sh debugimage
```

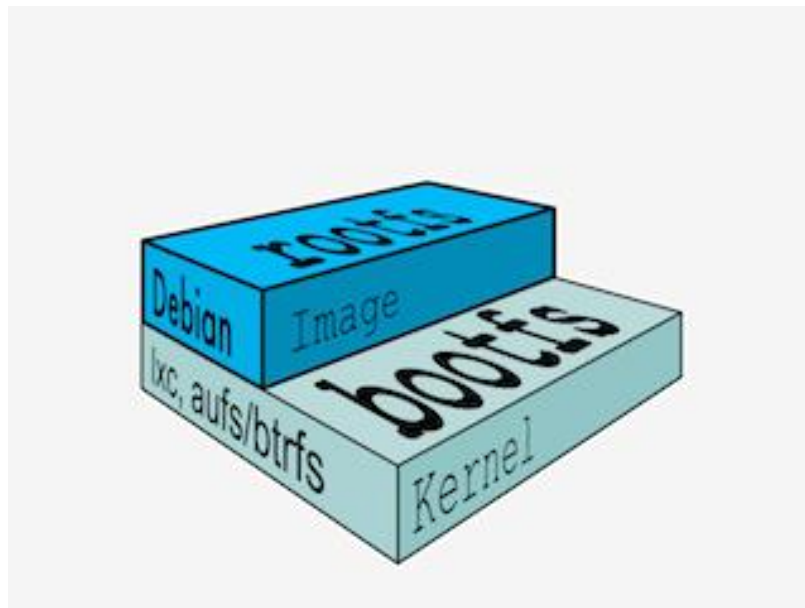
Obtaining a complete dump

- We can also dump the entire filesystem of a container.
- This is done with `docker export`.
- It generates a tar archive.

```
docker export <container_id> | tar tv
```

This will give a detailed listing of the content of the container.

Understanding Docker images



image

In this section, we will explain:

- What is an image.
- What is a layer.
- The various image namespaces.
- How to search and download images.
- Image tags and when to use them.

What is an image?

- Image = files + metadata
- These files form the root filesystem of our container.
- The metadata can indicate a number of things, e.g.:
 - the author of the image
 - the command to execute in the container when starting it
 - environment variables to be set
 - etc.
- Images are made of *layers*, conceptually stacked on top of each other.
- Each layer can add, change, and remove files and/or metadata.
- Images can share layers to optimize disk usage, transfer times, and memory use.

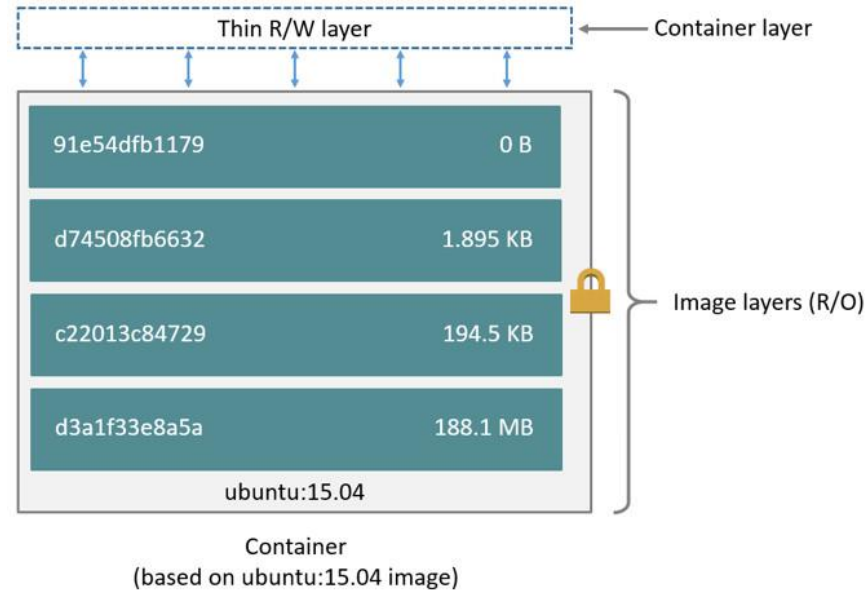
Example for a Java webapp

Each of the following items will correspond to one layer:

- CentOS base layer
- Packages and configuration files added by our local IT
- JRE
- Tomcat
- Our application's dependencies
- Our application code and assets
- Our application configuration

(Note: app config is generally added by orchestration facilities.)

The read-write layer

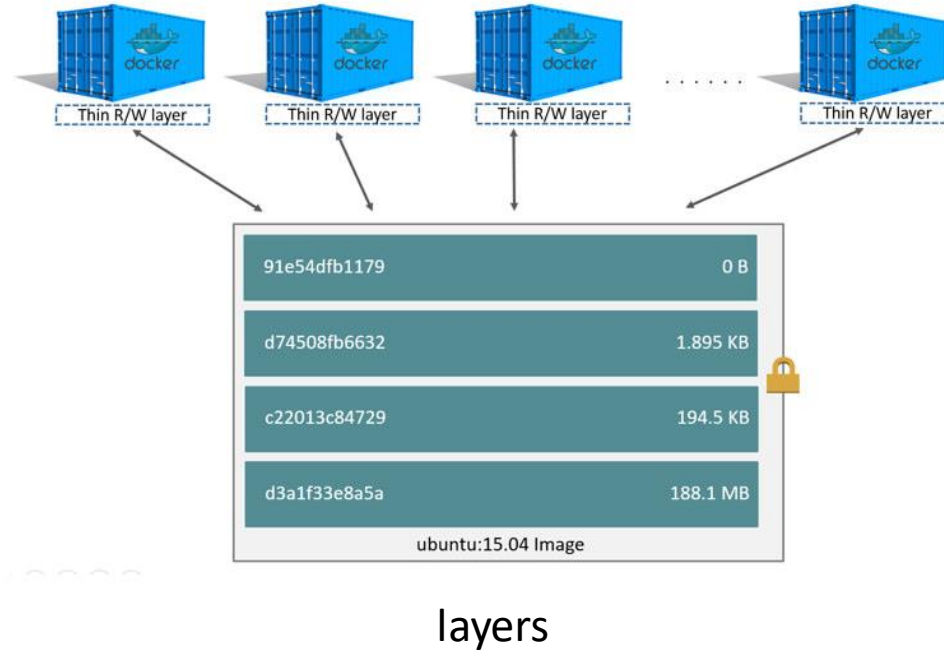


layers

Differences between containers and images

- An image is a read-only filesystem.
- A container is an encapsulated set of processes, running in a read-write copy of that filesystem.
- To optimize container boot time, *copy-on-write* is used instead of regular copy.
- `docker run` starts a container from a given image.

Multiple containers sharing the same image



Comparison with object-oriented programming

- Images are conceptually similar to *classes*.
- Layers are conceptually similar to *inheritance*.
- Containers are conceptually similar to *instances*.

If an image is read-only, how do we change it?

- We don't.
- We create a new container from that image.
- Then we make changes to that container.
- When we are satisfied with those changes, we transform them into a new layer.
- A new image is created by stacking the new layer on top of the old image.

A chicken-and-egg problem

- The only way to create an image is by “freezing” a container.
- The only way to create a container is by instantiating an image.
- Help!

There is a special empty image called `scratch`.

- It allows to *build from scratch*.

The `docker import` command loads a tarball into Docker.

- The imported tarball becomes a standalone image.
- That new image has a single layer.

Note: you will probably never have to do this yourself.

`docker commit`

- Saves all the changes made to a container into a new layer.
- Creates a new image (effectively a copy of the container).

`docker build` **(used 99% of the time)**

- Performs a repeatable build sequence.
- This is the preferred method!

We will explain both methods in a moment.

There are three namespaces:

- Official images

e.g. `ubuntu`, `busybox` ...

- User (and organizations) images

e.g. `jpetazzo/clock`

- Self-hosted images

e.g. `registry.example.com:5000/my-private/image`

Let's explain each of them.

The root namespace is for official images.

They are gated by Docker Inc.

They are generally authored and maintained by third parties.

Those images include:

- Small, “swiss-army-knife” images like busybox.
- Distro images to be used as bases for your builds, like ubuntu, fedora...
- Ready-to-use components and services, like redis, postgresql...
- Over 150 at this point!

The user namespace holds images for Docker Hub users and organizations.

For example:

`jpetazzo/clock`

The Docker Hub user is:

`jpetazzo`

The image name is:

`clock`

This namespace holds images which are not hosted on Docker Hub, but on third party registries.

They contain the hostname (or IP address), and optionally the port, of the registry server.

For example:

```
localhost:5000/wordpress
```

- `localhost:5000` is the host and port of the registry
- `wordpress` is the name of the image

Other examples:

```
quay.io/coreos/etcd
```

```
gcr.io/google-containers/hugo
```

How do you store and manage images?

Images can be stored:

- On your Docker host.
- In a Docker registry.

You can use the Docker client to download (pull) or upload (push) images.

To be more accurate: you can use the Docker client to tell a Docker Engine to push and pull images to and from a registry.

Showing current images

Let's look at what images are on our host now.

```
$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
fedora	latest	ddd5c9c1d0f2	3 days ago	204.7 MB
centos	latest	d0e7f81ca65c	3 days ago	196.6 MB
ubuntu	latest	07c86167cdc4	4 days ago	188 MB
redis	latest	4f5f397d4b7c	5 days ago	177.6 MB
postgres	latest	afe2b5e1859b	5 days ago	264.5 MB
alpine	latest	70c557e50ed6	5 days ago	4.798 MB
debian	latest	f50f9524513f	6 days ago	125.1 MB
busybox	latest	3240943c9ea3	2 weeks ago	1.114 MB
training/namer	latest	902673acc741	9 months ago	289.3 MB
jpetazzo/clock	latest	12068b93616f	12 months ago	2.433 MB

Searching for images

We cannot list *all* images on a remote registry, but we can search for a specific keyword:

```
$ docker search marathon
```

NAME	DESCRIPTION	STARS	OFFICIAL
AUTOMATED			
mesosphere/marathon	A cluster-wide init and co...	105	
[OK]			
mesoscloud/marathon	Marathon	31	
[OK]			
mesosphere/marathon-lb	Script to update haproxy b...	22	
[OK]			
tobilg/mongodb-marathon	A Docker image to start a ...	4	
[OK]			

- “Stars” indicate the popularity of the image.
- “Official” images are those in the root namespace.
- “Automated” images are built automatically by the Docker Hub. (This means that their build recipe is always available.)

There are two ways to download images.

- Explicitly, with `docker pull`.
- Implicitly, when executing `docker run` and the image is not found locally.

Pulling an image

```
$ docker pull debian:jessie
Pulling repository debian
b164861940b8: Download complete
b164861940b8: Pulling image (jessie) from debian
d1881793a057: Download complete
```

- As seen previously, images are made up of layers.
- Docker has downloaded all the necessary layers.
- In this example, `:jessie` indicates which exact version of Debian we would like.
It is a *version tag*.

- Images can have tags.
- Tags define image versions or variants.
- `docker pull ubuntu` will refer to `ubuntu:latest`.
- The `:latest` tag is generally updated often.

When to (not) use tags

Don't specify tags:

- When doing rapid testing and prototyping.
- When experimenting.
- When you want the latest version.

Do specify tags:

- When recording a procedure into a script.
- When going to production.
- To ensure that the same version will be used everywhere.
- To ensure repeatability later.

This is similar to what we would do with `pip install`, `npm install`, etc.

- An image can support multiple architectures
- More precisely, a specific *tag* in a given *repository* can have either:
 - a single *manifest* referencing an image for a single architecture
 - a *manifest list* (or *fat manifest*) referencing multiple images
- In a *manifest list*, each image is identified by a combination of:
 - `os` (linux, windows)
 - `architecture` (amd64, arm, arm64...)
 - optional fields like `variant` (for arm and arm64), `os.version` (for windows)

Working with multi-arch images

- The Docker Engine will pull “native” images when available
(images matching its own os/architecture/variant)
- We can ask for a specific image platform with `--platform`
- The Docker Engine can run non-native images thanks to QEMU+binfmt
(automatically on Docker Desktop; with a bit of setup on Linux)

We've learned how to:

- Understand images and layers.
- Understand Docker image namespacing.
- Search and download images.

In this section, we will create our first container image.

It will be a basic distribution image, but we will pre-install the package `figlet`.

We will:

- Create a container from a base image.
- Install software manually in the container, and turn it into a new image.
- Learn about new commands: `docker commit`, `docker tag`, and `docker diff`.

The plan

1. Create a container (with `docker run`) using our base distro of choice.
2. Run a bunch of commands to install and set up our software in the container.
3. (Optionally) review changes in the container with `docker diff`.
4. Turn the container into a new image with `docker commit`.
5. (Optionally) add tags to the image with `docker tag`.

Start an Ubuntu container:

```
$ docker run -it ubuntu  
root@<yourContainerId>: #/
```

Run the command `apt-get update` to refresh the list of packages available to install.

Then run the command `apt-get install figlet` to install the program we are interested in.

```
root@<yourContainerId>: #/ apt-get update && apt-get  
install figlet  
.... OUTPUT OF APT-GET COMMANDS ....
```

Type `exit` at the container prompt to leave the interactive session.
Now let's run `docker diff` to see the difference between the base image and our container.

```
$ docker diff <yourContainerId>
C /root
A /root/.bash_history
C /tmp
C /usr
C /usr/bin
A /usr/bin/figlet
...
```

As explained before:

- An image is read-only.
- When we make changes, they happen in a copy of the image.
- Docker can show the difference between the image, and its copy.
- For performance, Docker uses copy-on-write systems. (i.e. starting a container based on a big image doesn't incur a huge copy.)

Copy-on-write security benefits

- `docker diff` gives us an easy way to audit changes (à la Tripwire)
- Containers can also be started in read-only mode (their root filesystem will be read-only, but they can still have read-write data volumes)

Commit our changes into a new image

The `docker commit` command will create a new layer with those changes, and a new image using this new layer.

```
$ docker commit <yourContainerId>  
<newImageId>
```

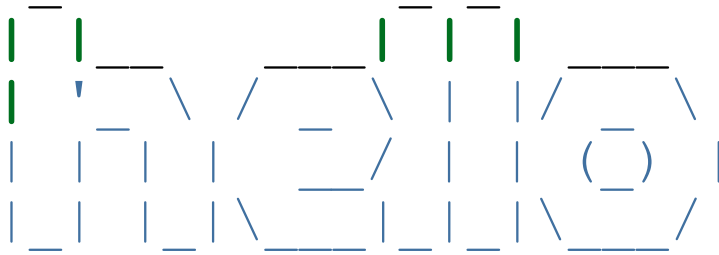
The output of the `docker commit` command will be the ID for your newly created image.

We can use it as an argument to `docker run`.

Testing our new image

Let's run this image:

```
$ docker run -it <newImageId>  
root@fcfb62f0bfde:/# figlet hello
```



It works! 🎉

Referring to an image by its ID is not convenient. Let's tag it instead.

We can use the `tag` command:

```
$ docker tag <newImageId> figlet
```

But we can also specify the tag as an extra argument to `commit`:

```
$ docker commit <containerId> figlet
```

And then run it using its tag:

```
$ docker run -it figlet
```

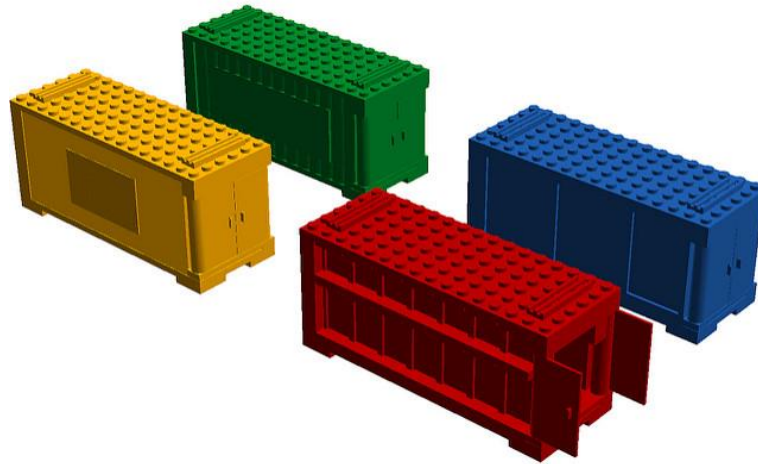
What's next?

Manual process = bad.

Automated process = good.

In the next chapter, we will learn how to automate the build process by writing a `Dockerfile`.

Building Docker images with a Dockerfile



Construction site with containers

Objectives

We will build a container image automatically, with a `Dockerfile`.

At the end of this lesson, you will be able to:

- Write a `Dockerfile`.
- Build an image from a `Dockerfile`.

- A `Dockerfile` is a build recipe for a Docker image.
- It contains a series of instructions telling Docker how an image is constructed.
- The `docker build` command builds an image from a `Dockerfile`.

Our Dockerfile must be in a **new, empty directory**.

1. Create a directory to hold our Dockerfile.

```
$ mkdir myimage
```

2. Create a Dockerfile inside this directory.

```
$ cd myimage
```

```
$ vim Dockerfile
```

Of course, you can use any other editor of your choice.

Type this into our Dockerfile...

```
FROM ubuntu
```

```
RUN apt-get update
```

```
RUN apt-get install figlet
```

- `FROM` indicates the base image for our build.
- Each `RUN` line will be executed by Docker during the build.
- Our `RUN` commands **must be non-interactive**. (No input can be provided to Docker during the build.)
- In many cases, we will add the `-y` flag to `apt-get`.

Save our file, then execute:

```
$ docker build -t figlet .
```

- `-t` indicates the tag to apply to the image.
- `.` indicates the location of the *build context*.

We will talk more about the build context later.

To keep things simple for now: this is the directory where our Dockerfile is located.

What happens when we build the image?

It depends if we're using BuildKit or not!

If there are lots of blue lines and the first line looks like this:

```
[+] Building 1.8s (4/6)
```

... then we're using BuildKit.

If the output is mostly black-and-white and the first line looks like this:

```
Sending build context to Docker daemon 2.048kB
```

... then we're using the “classic” or “old-style” builder.

Classic builder:

- copies the whole “build context” to the Docker Engine
- linear (processes lines one after the other)
- requires a full Docker Engine

BuildKit:

- only transfers parts of the “build context” when needed
- will parallelize operations (when possible)
- can run in non-privileged containers (e.g. on Kubernetes)

With the classic builder

The output of `docker build` looks like this:

```
docker build -t figlet .
Sending build context to Docker daemon  2.048kB
Step 1/3 : FROM ubuntu
----> f975c5035748
Step 2/3 : RUN apt-get update
----> Running in e01b294dbffd
(...output of the RUN command...)
Removing intermediate container e01b294dbffd
----> eb8d9b561b37
Step 3/3 : RUN apt-get install figlet
----> Running in c29230d70f9b
(...output of the RUN command...)
Removing intermediate container c29230d70f9b
----> 0dfd7a253f21
Successfully built 0dfd7a253f21
Successfully tagged figlet:latest
```

- The output of the `RUN` commands has been omitted.
- Let's explain what this output means.

Sending the build context to Docker

Sending build context to Docker daemon 2.048 kB

- The build context is the `.` directory given to `docker build`.
- It is sent (as an archive) by the Docker client to the Docker daemon.
- This allows to use a remote machine to build using local files.
- Be careful (or patient) if that directory is big and your link is slow.
- You can speed up the process with a [.dockerignore](#) file
 - It tells docker to ignore specific files in the directory
 - Only ignore files that you won't need in the build context!

Executing each step

Step 2/3 : RUN apt-get update

---> Running in e01b294dbffd

(...output of the RUN command...)

Removing intermediate container e01b294dbffd

---> eb8d9b561b37

- A container (e01b294dbffd) is created from the base image.
- The RUN command is executed in this container.
- The container is committed into an image (eb8d9b561b37).
- The build container (e01b294dbffd) is removed.
- The output of this step will be the base image for the next one.

```
[+] Building 7.9s (7/7) FINISHED
=> [internal] load build definition from Dockerfile                                0.0s
=> => transferring dockerfile: 98B                                                0.0s
=> [internal] load .dockerignore                                                  0.0s
=> => transferring context: 2B                                                    0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest                 1.2s
=> [1/3] FROM docker.io/library/ubuntu@sha256:cf31af331f38d1d7158470e095b132acd126a7180a54f263d386 3.2s
=> => resolve docker.io/library/ubuntu@sha256:cf31af331f38d1d7158470e095b132acd126a7180a54f263d386 0.0s
=> => sha256:cf31af331f38d1d7158470e095b132acd126a7180a54f263d386da88eb681d93 1.20kB / 1.20kB 0.0s
=> => sha256:1de4c5e2d8954bf5fa9855f8b4c9d3c3b97d1d380efe19f60f3e4107a66f5cae 943B / 943B 0.0s
=> => sha256:6a98cbe39225dadebcaa04e21dbe5900ad604739b07a9fa351dd10a6ebad4c1b 3.31kB / 3.31kB 0.0s
=> => sha256:80bc30679ac1fd798f3241208c14accd6a364cb8a6224d1127dfb1577d10554f 27.14MB / 27.14MB 2.3s
=> => sha256:9bf18fab4cfbf479fa9f8409ad47e2702c63241304c2cdd4c33f2a1633c5f85e 850B / 850B 0.5s
=> => sha256:5979309c983a2adef352538937475cf961d49c34194fa2aab142effe19ed9c1 189B / 189B 0.4s
=> => extracting sha256:80bc30679ac1fd798f3241208c14accd6a364cb8a6224d1127dfb1577d10554f 0.7s
=> => extracting sha256:9bf18fab4cfbf479fa9f8409ad47e2702c63241304c2cdd4c33f2a1633c5f85e 0.0s
=> => extracting sha256:5979309c983a2adef352538937475cf961d49c34194fa2aab142effe19ed9c1 0.0s
=> [2/3] RUN apt-get update                                                       2.5s
=> [3/3] RUN apt-get install figlet                                              0.9s
=> exporting to image                                                            0.1s
=> => exporting layers                                                            0.1s
=> => writing image sha256:3b8aee7b444ab775975dfba691a72d8ac24af2756e0a024e056e3858d5a23f7c 0.0s
=> => naming to docker.io/library/figlet                                         0.0s
```

Understanding BuildKit output

- BuildKit transfers the Dockerfile and the *build context* (these are the first two `[internal]` stages)
- Then it executes the steps defined in the Dockerfile (`[1/3]`, `[2/3]`, `[3/3]`)
- Finally, it exports the result of the build (image definition + collection of layers)

BuildKit plain output

- When running BuildKit in e.g. a CI pipeline, its output will be different
- We can see the same output format by using `--progress=plain`

If you run the same build again, it will be instantaneous. Why?

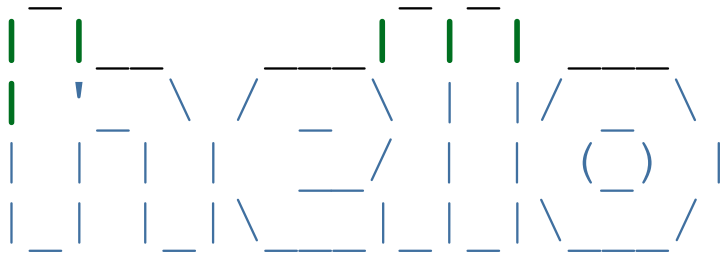
- After each build step, Docker takes a snapshot of the resulting image.
- Before executing a step, Docker checks if it has already built the same sequence.
- Docker uses the exact strings defined in your Dockerfile, so:
 - `RUN apt-get install figlet cowsay` is different from `RUN apt-get install cowsay figlet`
 - `RUN apt-get update` is not re-executed when the mirrors are updated

You can force a rebuild with `docker build --no-cache . . . .`

Running the image

The resulting image is not different from the one produced manually.

```
$ docker run -ti figlet  
root@91f3c974c9a1:/# figlet hello
```



Yay! 🎉

Using image and viewing history

The `history` command lists all the layers composing an image.

For each layer, it shows its creation time, size, and creation command.

When an image was built with a Dockerfile, each layer corresponds to a line of the Dockerfile.

```
$ docker history figlet
```

IMAGE	CREATED	CREATED BY	SIZE
f9e8f1642759	About an hour ago	/bin/sh -c apt-get install fi	1.627 MB
7257c37726a1	About an hour ago	/bin/sh -c apt-get update	21.58 MB
07c86167cdc4	4 days ago	/bin/sh -c <i>\$(nop) CMD ["/bin</i>	<i>0 B</i>
<missing>	4 days ago	/bin/sh -c sed -i 's/^#\s*\('	1.895 kB
<missing>	4 days ago	/bin/sh -c echo '#!/bin/sh'	194.5 kB
<missing>	4 days ago	/bin/sh -c <i>\$(nop) ADD file:b</i>	187.8 MB

- On UNIX, to start a new program, we need two system calls:
 - `fork()`, to create a new child process;
 - `execve()`, to replace the new child process with the program to run.
- Conceptually, `execve()` works like this:
`execve(program, [list, of, arguments])`
- When we run a command, e.g. `ls -l /tmp`, something needs to parse the command.
(i.e. split the program and its arguments into a list.)
- The shell is usually doing that.
(It also takes care of expanding environment variables and special things like `~`.)

Why `sh -c`?

- When we do `RUN ls -l /tmp`, the Docker builder needs to parse the command.
- Instead of implementing its own parser, it outsources the job to the shell.
- That's why we see `sh -c ls -l /tmp` in that case.
- But we can also do the parsing jobs ourselves.
- This means passing `RUN` a list of arguments.
- This is called the *exec syntax*.

Dockerfile commands that execute something can have two forms:

- plain string, or *shell syntax*: `RUN apt-get install figlet`
- JSON list, or *exec syntax*: `RUN ["apt-get", "install", "figlet"]`

We are going to change our Dockerfile to see how it affects the resulting image.

Let's change our Dockerfile as follows!

```
FROM ubuntu
RUN apt-get update
RUN ["apt-get", "install", "figlet"]
```

Then build the new Dockerfile.

```
$ docker build -t figlet .
```

History with exec syntax

Compare the new history:

```
$ docker history figlet
```

IMAGE	CREATED	CREATED BY	SIZE
27954bb5faaf	10 seconds ago	apt-get install figlet	1.627 MB
7257c37726a1	About an hour ago	/bin/sh -c apt-get update	21.58 MB
07c86167cdc4	4 days ago	/bin/sh -c #(nop) CMD ["/bin	0 B
<missing>	4 days ago	/bin/sh -c sed -i 's/^#\s*\('	1.895 kB
<missing>	4 days ago	/bin/sh -c echo '#!/bin/sh'	194.5 kB
<missing>	4 days ago	/bin/sh -c #(nop) ADD file:b	187.8 MB

- Exec syntax specifies an *exact* command to execute.
- Shell syntax specifies a command to be wrapped within `/bin/sh -c "..."`.

When to use exec syntax and shell syntax

- shell syntax:
 - is easier to write
 - interpolates environment variables and other shell expressions
 - creates an extra process (`/bin/sh -c ...`) to parse the string
 - requires `/bin/sh` to exist in the container
- exec syntax:
 - is harder to write (and read!)
 - passes all arguments without extra processing
 - doesn't create an extra process
 - doesn't require `/bin/sh` to exist in the container

POSIX shells have a built-in command named `exec`.

`exec` should be followed by a program and its arguments.

From a user perspective:

- it looks like the shell exits right away after the command execution,
- in fact, the shell exits just *before* command execution;
- or rather, the shell gets *replaced* by the command.

Example using `exec`

```
CMD exec figlet -f script hello
```

In this example, `sh -c` will still be used, but `figlet` will be PID 1 in the container.

The shell gets replaced by `figlet` when `figlet` starts execution.

This allows to run processes as PID 1 without using JSON.

CMD and ENTRYPOINT



Container entry doors

In this lesson, we will learn about two important Dockerfile commands: `CMD` and `ENTRYPOINT`.

These commands allow us to set the default command to run in a container.

Defining a default command

When people run our container, we want to greet them with a nice hello message, and using a custom font.

For that, we will execute:

```
figlet -f script hello
```

- `-f script` tells figlet to use a fancy font.
- `hello` is the message that we want it to display.

Our new Dockerfile will look like this:

```
FROM ubuntu
RUN apt-get update
RUN ["apt-get", "install", "figlet"]
CMD figlet -f script hello
```

- CMD defines a default command to run when none is given.
- It can appear at any point in the file.
- Each CMD will replace and override the previous one.
- As a result, while you can have multiple CMD lines, it is useless.

Build and test our image

Let's build it:

```
$ docker build -t figlet .
```

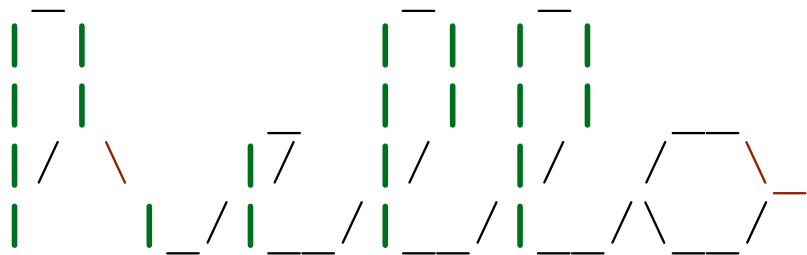
...

```
Successfully built 042dff3b4a8d
```

```
Successfully tagged figlet:latest
```

And run it:

```
$ docker run figlet
```



If we want to get a shell into our container (instead of running `figlet`), we just have to specify a different program to run:

```
$ docker run -it figlet bash  
root@7ac86a641116:/#
```

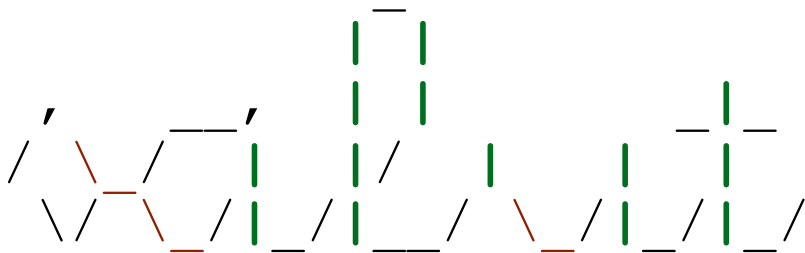
- We specified `bash`.
- It replaced the value of `CMD`.

Using ENTRYPOINT

We want to be able to specify a different message on the command line, while retaining `figlet` and some default parameters.

In other words, we would like to be able to do this:

```
$ docker run figlet salut
```



We will use the `ENTRYPOINT` verb in Dockerfile.

Adding ENTRYPOINT to our Dockerfile

Our new Dockerfile will look like this:

```
FROM ubuntu
RUN apt-get update
RUN ["apt-get", "install", "figlet"]
ENTRYPOINT ["figlet", "-f", "script"]
```

- ENTRYPOINT defines a base command (and its parameters) for the container.
- The command line arguments are appended to those parameters.
- Like CMD, ENTRYPOINT can appear anywhere, and replaces the previous value.

Why did we use JSON syntax for our ENTRYPOINT?

Implications of JSON vs string syntax

- When CMD or ENTRYPOINT use string syntax, they get wrapped in `sh -c`.
- To avoid this wrapping, we can use JSON syntax.

What if we used ENTRYPOINT with string syntax?

```
$ docker run figlet salut
```

This would run the following command in the `figlet` image:

```
sh -c "figlet -f script" salut
```

Build and test our image

Let's build it:

```
$ docker build -t figlet .
```

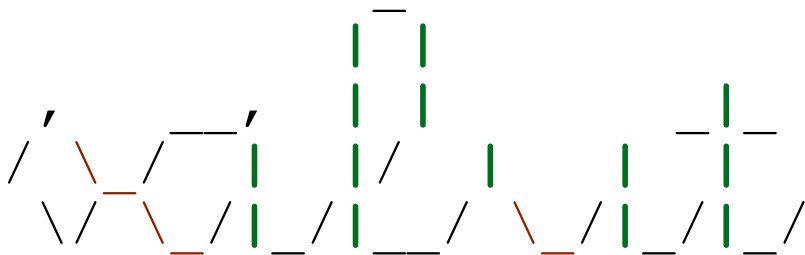
...

```
Successfully built 36f588918d73
```

```
Successfully tagged figlet:latest
```

And run it:

```
$ docker run figlet salut
```



What if we want to define a default message for our container?

Then we will use `ENTRYPOINT` and `CMD` together.

- `ENTRYPOINT` will define the base command for our container.
- `CMD` will define the default parameter(s) for this command.
- They *both* have to use JSON syntax.

Our new Dockerfile will look like this:

```
FROM ubuntu
RUN apt-get update
RUN ["apt-get", "install", "figlet"]
ENTRYPOINT ["figlet", "-f", "script"]
CMD ["hello world"]
```

- ENTRYPOINT defines a base command (and its parameters) for the container.
- If we don't specify extra command-line arguments when starting the container, the value of CMD is appended.
- Otherwise, our extra command-line arguments are used instead of CMD.

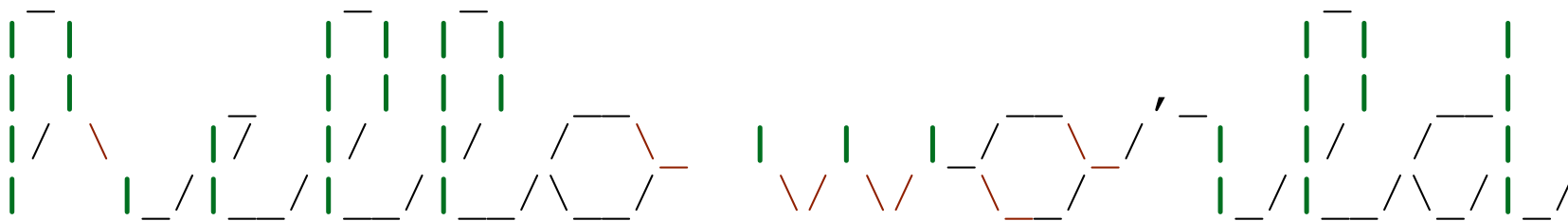
Build and test our image

Let's build it:

```
$ docker build -t myfiglet .  
...  
Successfully built 6e0b6a048a07  
Successfully tagged myfiglet:latest
```

Run it without parameters:

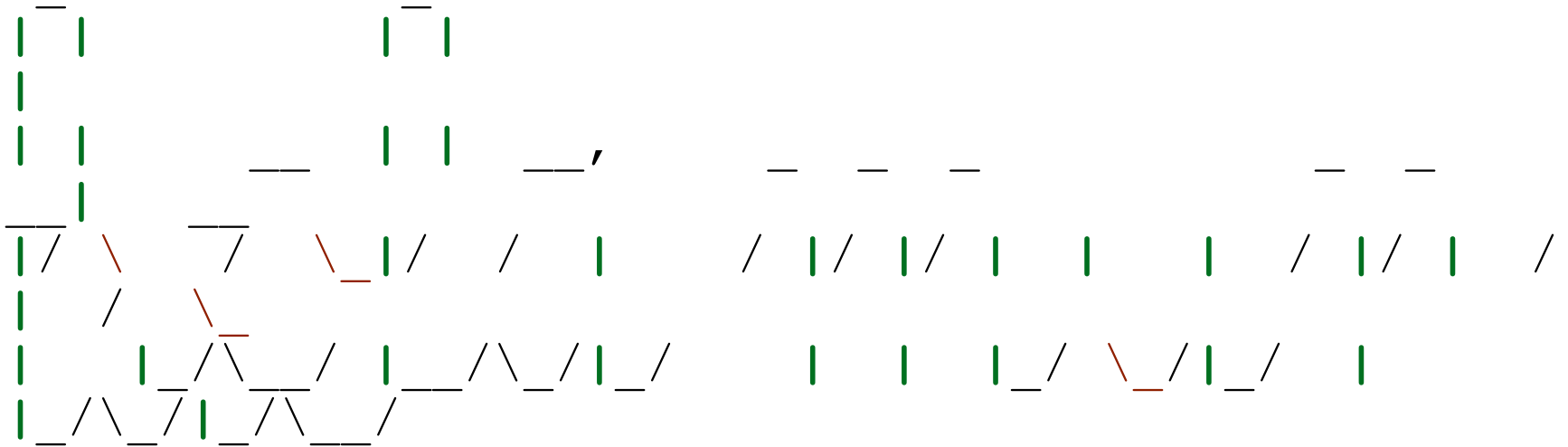
```
$ docker run myfiglet
```



Overriding the image default parameters

Now let's pass extra arguments to the image.

```
$ docker run myfiglet hola mundo
```



We overrode CMD but still used ENTRYPOINT.

What if we want to run a shell in our container?

We cannot just do `docker run myfiglet bash` because that would just tell figlet to display the word “bash.”

We use the `--entrypoint` parameter:

```
$ docker run -it --entrypoint bash myfiglet  
root@6027e44e2955: /#
```

CMD and ENTRYPOINT recap

- `docker run myimage` **executes** `ENTRYPOINT + CMD`
- `docker run myimage args` **executes** `ENTRYPOINT + args` (overriding `CMD`)
- `docker run --entrypoint prog myimage` **executes** `prog` (overriding both)

Command	ENTRYPOINT	CMD	Result
<code>docker run figlet</code>	<code>none</code>	<code>none</code>	Use values from base image (bash)
<code>docker run figlet hola</code>	<code>none</code>	<code>none</code>	Error (executable hola not found)
<code>docker run figlet figlet -f script</code>	<code>none</code>	<code>figlet -f script</code>	
<code>docker run figlet hola figlet -f script</code>	<code>none</code>	<code>figlet -f script</code>	
<code>docker run figlet none figlet -f script</code>	<code>figlet -f script</code>	<code>figlet -f script</code>	
<code>docker run figlet hola none figlet -f script</code>	<code>figlet -f script</code>	<code>figlet -f script</code>	Error (executable hola not found)
<code>docker run figlet figlet -f script hello</code>	<code>figlet -f script</code>	<code>hello</code>	
<code>docker run figlet hola figlet -f script hello</code>	<code>figlet -f script</code>	<code>hello</code>	
<code>docker run figlet none figlet -f script hola</code>	<code>figlet -f script</code>	<code>hola</code>	

When to use ENTRYPOINT vs CMD

ENTRYPOINT is great for “containerized binaries”.

Example: `docker run consul --help`

(Pretend that the `docker run` part isn't there!)

CMD is great for images with multiple binaries.

Example: `docker run busybox ifconfig`

(It makes sense to indicate *which* program we want to run!)

???

:EN:- CMD and ENTRYPOINT :FR:- CMD et ENTRYPOINT

Copying files during the build



Monks copying books

So far, we have installed things in our container images by downloading packages.

We can also copy files from the *build context* to the container that we are building.

Remember: the *build context* is the directory containing the Dockerfile. In this chapter, we will learn a new Dockerfile keyword: `COPY`.

We want to build a container that compiles a basic “Hello world” program in C.

Here is the program, `hello.c`:

```
int main () {  
    puts("Hello, world!");  
    return 0;  
}
```

Let's create a new directory, and put this file in there.

Then we will write the Dockerfile.

On Debian and Ubuntu, the package `build-essential` will get us a compiler.

When installing it, don't forget to specify the `-y` flag, otherwise the build will fail (since the build cannot be interactive).

Then we will use `COPY` to place the source file into the container.

```
FROM ubuntu
RUN apt-get update
RUN apt-get install -y build-essential
COPY hello.c /
RUN make hello
CMD /hello
```

Create this Dockerfile.



Testing our C program

- Create `hello.c` and `Dockerfile` in the same directory.
 - Run `docker build -t hello .` in this directory.
 - Run `docker run hello`, you should see `Hello, world!`.
- Success!

- Run the build again.
- Now, modify `hello.c` and run the build again.
- Docker can cache steps involving `COPY`.
- Those steps will not be executed again if the files haven't been changed.

- We can `COPY` whole directories recursively
- It is possible to do e.g. `COPY . .`
(but it might require some extra precautions to avoid copying too much)
- In older Dockerfiles, you might see the `ADD` command; consider it deprecated
(it is similar to `COPY` but can automatically extract archives)
- If we really wanted to compile C code in a container, we would:
 - place it in a different directory, with the `WORKDIR` instruction
 - even better, use the `gcc` official image

- We can create a file named `.dockerignore`
(at the top-level of the build context)
- It can contain file names and globs to ignore
- They won't be sent to the builder
(and won't end up in the resulting image)
- See the [documentation](#) for the little details
(exceptions can be made with `!`, multiple directory levels with `**...`)

Let's write Dockerfiles for an existing application!

1. Check out the code repository
2. Read all the instructions
3. Write Dockerfiles
4. Build and test them individually

Clone the repository available at:

<https://github.com/jpetazzo/wordsmith>

It should look like this:

```
|— LICENSE
|— README
|— db/
|   └─ words.sql
|— web/
|   └─ dispatcher.go
|       └─ static/
└─ words/
    └─ pom.xml
```

Instructions

The repository contains instructions in English and French. For now, we only care about the first part (about writing Dockerfiles). Place each Dockerfile in its own directory, like this:

```
├-- LICENSE
├-- README
├-- db/
|   ├── `Dockerfile`
|   └-- words.sql
├-- web/
|   ├── `Dockerfile`
|   ├── dispatcher.go
|   └-- static/
└-- words/
    ├── `Dockerfile`
    └-- pom.xml
```

Build and run each Dockerfile individually.

For `db`, we should be able to see some messages confirming that the data set was loaded successfully (some `INSERT` lines in the container output).

For `web` and `words`, we should be able to see some message looking like “server started successfully”.

That’s all we care about for now!

Bonus question: make sure that each container stops correctly when hitting Ctrl-C.

???

Place the following Compose file at the root of the repository:

```
version: "3"
services:
  db:
    build: db
  words:
    build: words
  web:
    build: web
    ports:
      - 8888:80
```

Test the whole app by bringin up the stack and connecting to port 8888.